Supplementary Material

A Data and Source Code Availability

Models' responses are stored in the pickle file of each model, and embedding trajectories can be found in our anonymized drive due to size limit. Scripts for collecting the results can also be found there.

B Computational Hardware and Cloud APIs

Data collection for open-source models was conducted using a computing cluster equipped with seven NVIDIA L40s GPUs, as well as Google Cloud instances featuring A100 GPUs. For proprietary models, responses were obtained through official APIs provided by OpenAI, while Gemini model outputs were collected using the Vertex AI platform.

C Creation of Testing Sequences

C.1 Word selection

We first selected a set of words that are 5 letters long, and we created both homogeneous and heterogeneous sequences of words by randomly sampling from this list. A critical prerequisite for a valid analysis of counting behavior and its underlying neural dynamics is to ensure that the linguistic units of counting are represented as single tokens within the model's vocabulary. To this end, we verified the tokenizer for each model in our study to confirm that all target words (e.g. `'table'` in the previous example) were encoded as single tokens. This control ensures that the observed neural dynamics reflect the cognitive process of counting and are not artifacts of sub-word tokenization or compositional encoding.

C.2 Homogeneous stimuli

The homogeneous stimuli were created by randomly sampling a word from the curated list or a random letter, then repeating it the target number of times for the naming task. In the production task, the model was asked to write the sampled word or letter the target number of times.

C.3 Heterogeneous stimuli

The non-uniform target string in this case was formed by different letters or words. For the naming task, both sequences of letters and words are tested, while

for the production task, only heterogeneous letters were asked [2]. Rather than sampling one word and repeating it the target number of times, we sampled the target number of different words and formed a string. While for the letters, repetition of the same letter is allowed. Results with heterogeneous stimuli can be found in Section G.4.

D Prompting Details

D.1 Response Formatting

As discussed in Section 3.3, our system message consists of four components: (1) *contextual background*, (2) *task goal*, (3) *counting strategy*, and (4) *response formatting*. To maximize the performance of each LLM and ensure that responses adhere to the desired format, we specifically investigated the design of the *response formatting* component for models that exhibited lower proportions of valid outputs. These models include Llama-3B and Llama-8B, which underperformed compared to others in terms of valid trial rates. For reference, the proportions of valid trials achieved by the higher-performing models are as follows: Gemini 2.5 Pro achieved 98.3%, GPT-5 and GPT-4.1 achieved 100.0%, Llama-70B achieved 93.3% and Qwen-32B achieved 87.5%.

To optimize the response formatting instructions, we explored several strategies: (1) varying the position of the response formatting (either preceding or following the task goal); (2) experimenting with alternative phrasings of the instruction; and (3) providing an explicit example of a correctly formatted response. We conducted a controlled experiment to evaluate the effect of each variation. For both the naming and production tasks, we executed 10 trials per numerosity. Each set of 10 trials consisted of 5 trials using words and 5 using letters, both sampled randomly. The response formatting instruction that yielded the highest proportion of valid outputs was selected.

The alternative phrasings we explored are:

1. Format/write/give your response between an opening tag <my_answer> and a close tag </my_answer>.
2. Please format all your answers by wrapping them in a <my_answer> opening tag and a </my_answer> closing tag.
3. Begin/start answering with <my_answer> and finish/stop with </my_answer>.

For Llama-3B, the best response formatting was "Format your response between an opening tag <my_answer> and a close tag </my_answer>" without changing position and giving explicit examples. For Llama-8B, "Please format all your answers by wrapping them in a <my_answer> opening tag and a

[2] Heterogeneous word generation poses a significant challenge to our automatic response parsing pipeline. Even with human inspection, tricky cases still exist. For instance, "OK, I will start generating 20 random words: apple, banana, cat, ice cream....". On the other hand, for letters, we explicitly asked the model to generate a space before each letter, which can be parsed easily with our pipeline.

</my_answer> closing tag. For example, if the answer is '76', your response should be <my_answer>76</my_answer>." resulted in giving more valid trials. After applying those response formatting, Llama-3B has 75.8% of valid trials and Llama-8B has 83.5% of valid trials.

D.2 Prompt Examples

We provide a more complete set of per-trial prompts in the shared drive.

E Inference Details

E.1 Token Limits

To accelerate the data collection process and minimize computational overhead, we imposed upper limits on the number of tokens generated by the models. Specifically, for the naming task, the maximum number of tokens allowed was set to 512 for the word condition and 256 for the letter condition. For the production task, the corresponding limits were 1024 tokens for the word condition and 512 tokens for the letter condition. Those limits are further adjusted during generation (details in Section F). For Qwen-32B, those token limits apply to the tokens after the '</think>' token, and we set a 2048 token limit for the reasoning process to prevent a non-stopping think loop.

E.2 Tokenization Control

To examine internal model representations, we extracted the hidden states from the final layer of the Llama-70B model, which consists of 8192 neurons per token. These activations provide a high-dimensional embedding of the model's internal processing during generation.

To isolate representational patterns independent of behavioral outcomes, we conducted a separate experiment from the main analyses presented in Section 4.1 and 4.2. In this auxiliary run, we selected a single concrete word, "apple," and focused exclusively on the production task. For each numerosity level, we generated at least 10 valid trials. This approach allowed for controlled analysis by eliminating the variability introduced by the tokenization of letter sequences. Unlike words, letter sequences (e.g., "M," "MM," and "MMMM" are all represented by a single token) are subject to inconsistent tokenization. By using a single, concrete word, we ensured stable token boundaries and reliable extraction of hidden states.

F Automatic Response Parsing Pipeline

Each trial was checked for the presence of the special tags `<my_answer>` and `</my_answer>`. If missing, the trial was marked *invalid*, and the model was allowed up to nine additional attempts (ten total) to produce a valid response. In the

production task, to prevent non-terminating output (e.g., repetition), we enforced task-specific token limits. If the model reached this limit, the token budget was halved and retried until it dropped to 512 tokens. Hitting the 512-token ceiling on three consecutive attempts rendered the trial invalid; otherwise, up to ten retries were allowed.

Post-generation, valid responses were parsed based on task type. For the naming task, we extracted content between the answer tags and checked if it contained a single integer. Non-integer outputs were flagged for manual review. In the production task, we used regular expressions to extract and count only target tokens (letters or words) within the tags; extraneous text also triggered manual inspection. We made sure that spelling errors in the generated words were not regarded as wrong responses by implementing a post-processing pipeline measuring edit (Levenshtein) distance to find cases where the output word is only moderately different from the target word, and making sure that these were counted as correct trials. The results showed that all words generated have 0 distance from the target word, showcasing no spelling errors at all.

G Results

G.1 Behavior Patterns

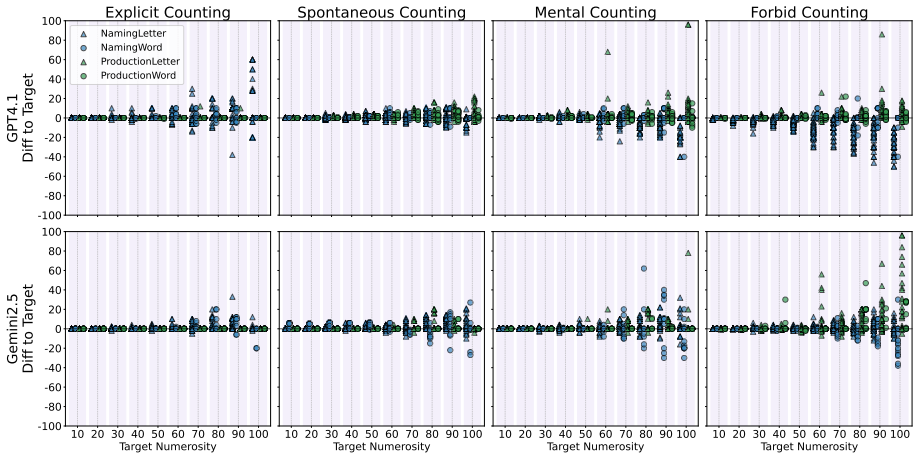

Fig. 6. Scatter plots of enumeration errors (response – target value) across different prompting conditions for a selected subset of models. Each panel shows the response differences plotted against the target values, with distinct markers of different colors representing different task-stimulus combinations: blue for the naming task and green for the production task; triangles for letters and filled circles for words.

Systematic Counting We investigated whether the models engaged in systematic counting when required (explicit counting) and when contextually beneficial (spontaneous counting). This analysis focused on the production task. The naming task was excluded, as the models frequently returned errors, potentially due to either repeating the target string incorrectly or failing to count. The current study design does not allow us to disentangle these factors.

In the explicit counting condition, all models systematically counted on every valid trial. GPT-5 and Gemini 2.5 produced error-free responses in all valid cases. GPT-4.1 made 3 errors, Llama70B made 1 error, and Qwen-32B made 57 errors. Of GPT-4.1's errors, two were due to grouping instead of systematic counting. Both GPT-4.1 and Llama-70B made an error on the word *under*, despite counting systematically; in both cases, they under-enumerated, suggesting that semantic information may interfere with counting accuracy. Qwen-32B's errors arose not from a failure to count but from mistakes during the copying of letters and words from the think loop to our special tags without systematic counting again inside the special answer tags. In the spontaneous counting condition, among all the results, only Gemini 2.5 Pro deployed systematic counting in the production task for one trial (ID. 5385).

Examples of Counting Behavior We provide some representative examples of models' counting behaviors. Full responses can be found in the pickle files we provide.

1. **Grouping** — *Gemini 2.5 Pro Production ID:197*
   "Okay, I will write the letter J exactly 70 times, using numbered groups to keep track.
   1 J J J J J J J J J J (10)
   2 J J J J J J J J J J (20)
   3 J J J J J J J J J J (30)
   4 J J J J J J J J J J (40)
   5 J J J J J J J J J J (50)
   6 J J J J J J J J J J (60)
   7 J J J J J J J J J J (70)
   I have written the letter J exactly 70 times."
2. **Systematic Counting** — *Gemini 2.5 Pro Production ID:5*
   "1. Q
   2. Q
   3. Q
   4. Q
   5. Q
   6. Q
   7. Q
   8. Q
   9. Q
   10. Q"

3. **Grouping and Systematic Counting** — *Gemini 2.5 Pro Production ID:581*
"Okay, I will write the letter U exactly 90 times. I will use numbering in groups of 10 to keep track.
1 2 3 4 5 6 7 8 9 10
U U U U U U U U U U (10)
11 12 13 14 15 16 17 18 19 20
U U U U U U U U U U (20)
21 22 23 24 25 26 27 28 29 30
U U U U U U U U U U (30)
31 32 33 34 35 36 37 38 39 40
U U U U U U U U U U (40)
41 42 43 44 45 46 47 48 49 50
U U U U U U U U U U (50)
51 52 53 54 55 56 57 58 59 60
U U U U U U U U U U (60)
61 62 63 64 65 66 67 68 69 70
U U U U U U U U U U (70)
71 72 73 74 75 76 77 78 79 80
U U U U U U U U U U (80)
81 82 83 84 85 86 87 88 89 90
U U U U U U U U U U (90)
I have written the letter U exactly 90 times."
4. **Wrong Repeating and Correct Counting** — *GPT-4.1 Naming ID:405*
"Let me repeat the string and use markers to count:
u u u u u u u u u u u u u u u u u u u u u u u u u u u u u u u u
Now, let's count by grouping into 5s for clarity:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32
Final count: 32"
5. **Correct Repeating but Wrong Counting** — *GPT-4.1 Naming ID:500*
"Let's repeat the string:
storm storm storm storm storm storm storm storm storm storm
storm storm storm storm storm storm storm storm storm storm
storm storm storm storm storm storm storm storm storm storm
storm storm storm storm storm storm storm storm storm storm
storm storm storm storm storm storm storm storm storm storm
storm storm storm storm storm storm storm storm storm storm
storm storm storm storm storm storm storm storm storm storm

Now, let's count the words using markers (I'll use '|' for every 10 counts):
storm storm storm storm storm storm storm storm storm storm | (10)
storm storm storm storm storm storm storm storm storm storm | (20)
storm storm storm storm storm storm storm storm storm storm | (30)
storm storm storm storm storm storm storm storm storm storm | (40)

storm storm storm storm storm storm storm storm storm storm | (50)
storm storm storm storm storm storm storm storm storm storm | (60)
Each line is 10, and there are 7 lines. Let's check: 1st line: 10 2nd line: 10 3rd line: 10 4th line: 10 5th line: 10 6th line: 10 7th line: 10
$10 + 10 + 10 + 10 + 10 + 10 + 10 = 70$
Final count: 70"

G.2 Accuracy and MAE with Model Sizes

We investigated the effect of model scaling on sequential enumeration tasks using models from the Llama family, specifically Llama-3B, Llama-8B, and Llama-70B. We selected the Llama family for this comparison because these models share the same architecture and training objective, differing only in model size, that is, primarily through the number of layers and hidden units.

We evaluated performance using two metrics: accuracy and mean absolute error (MAE). As shown in Figure 7, both accuracy and MAE improve progressively with model size. This indicates that the ability to perform sequential enumeration does not exhibit a sharp emergence but rather improves gradually as model capacity increases within the tested size range. These findings suggest that sequential enumeration is not an emergent ability in the strict sense, but instead scales smoothly with model size in this family.

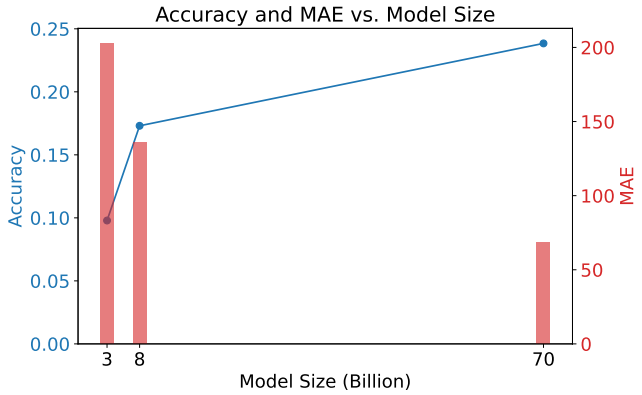

Fig. 7. Comparison of model performance across different Llama model sizes. Accuracy is shown as a blue line plot (left y-axis), and Mean Absolute Error (MAE) is represented by red bars (right y-axis). As model size increases from 3B to 70B, accuracy generally improves while MAE decreases, indicating better overall performance with larger models.

G.3 Friedman Test

We conducted a comprehensive statistical analysis to compare the performance of four different counting conditions (explicit, spontaneous, mental, and forbid)

across five models using Normalized Absolute Error (NAE) as the primary metric. NAE is calculated as the absolute difference between the target count and the model's predicted count, normalized by the target count: NAE = |target - predicted| / target. This metric provides a scale-invariant measure of counting accuracy, where values closer to 0 indicate better performance. We employed the non-parametric Friedman test since the data did not meet normality assumptions. Effect sizes were quantified using Kendall's W for the overall test and rank-biserial correlation for pairwise comparisons. When significant differences were detected, post-hoc pairwise Mann-Whitney U tests with Bonferroni correction ($\alpha = 0.0083$) were conducted to identify specific method differences.

The statistical analysis revealed significant differences in counting performance across methods for all five models, though with varying effect sizes and patterns. Table 1 shows the NAE across different models and counting methods. GPT-5 showed the best performance by being error-free in the explicit word production task. The overall effect size is small (W = 0.02), and Post-hoc pairwise comparisons with a Bonferroni correction revealed that the source of this significance stemmed specifically from the 'explicit' method. GPT-4.1 demonstrated the second-best overall performance with consistently low NAE values (ranging from 0.02 to 0.08) and showed a medium effect size (W = 0.14), with explicit counting significantly outperforming all other methods. Gemini 2.5 Pro exhibited similarly strong performance (NAE values 0.07-0.17) but with a smaller effect size (W = 0.05), showing that explicit counting was significantly better than spontaneous, mental, and forbid methods. In contrast, Llama 70B displayed the most pronounced method differences with a large effect size (W = 0.37) and substantially higher NAE values, where explicit counting (NAE = 0.21) significantly outperformed all other methods, with performance deteriorating progressively through spontaneous (0.69), mental (1.03), to forbid (1.32). Qwen 32B showed an interesting pattern where mental (0.90) and forbid (0.79) methods performed better than explicit (1.38) and spontaneous (1.71) methods, though the overall effect size remained small (W = 0.03).

In summary, our findings demonstrate that the counting method significantly affects performance across all tested language models, with GPT-4.1 and Gemini 2.5 Pro showing superior accuracy and consistent preference for explicit counting, while Llama 70B exhibited the strongest method sensitivity, and Qwen 32B uniquely benefited from implicit counting approaches.

G.4 Heterogeneous Stimuli

We also tested how models perform when the stimuli are heterogeneous (*i.e.,* a string of random words or letters). For the production task, only random letters are tested, as it is challenging to find a reliable way to automate the answer validation process. As shown in Fig. 8, similar trends can be observed as when the models were tested with homogeneous stimuli. Notably, GPT-5 delivered perfect responses in the explicit and spontaneous conditions. Qwen was flagged as having an insufficient number of valid trials.

Table 1. NAE summary statistics by model and counting method

Model	Explicit	Spontaneous	Mental	Forbid
GPT5	0.05 ± 0.10	0.05 ± 0.08	0.05 ± 0.09	0.06 ± 0.10
GPT4.1	0.02 ± 0.06	0.03 ± 0.04	0.04 ± 0.07	0.08 ± 0.11
Gemini2.5pro	0.07 ± 0.66	0.08 ± 0.53	0.09 ± 0.58	0.17 ± 0.90
Llama70b	0.21 ± 0.71	0.69 ± 1.24	1.03 ± 1.58	1.32 ± 1.91
Qwen32b	1.38 ± 3.33	1.71 ± 3.31	0.90 ± 2.31	0.79 ± 2.07

G.5 Counting via coding

To investigate whether LLMs' coding ability could support perfect sequential enumeration, we select the SOTA coding LLM, GPT-5 as the testing model (see Aider LLM Leaderboards). As generating letters or words, either uniformly or non-uniformly, is too easy, we focus only on the non-uniform naming task. We gave 30 trials for GPT-5 to generate a code snippet that can count the number of letters or words in a given string. The prompt is:

```
You need to write a code snippet to count how many letters or
words in a given string. The input is a string and the output
of the snippet should be an integer number. Try your best to
consider all possible conditions and return the final Python
code.
```

After human inspection and testing the code snippets, our results revealed that only 11 trials (36%) are suitable for a string that could be either words or letters. Other code snippets require specification of whether the goal is to count words or letters. Among those valid trials, 10 trials returned perfect accuracy, and one code snippet achieved 50% of accuracy. We provide the generated code snippet in the drive folder.

G.6 Analysis of Embeddings

Figure 9 illustrates the evolution of the first five PCs over the course of generation steps across the four counting conditions. Notably, the mental and forbid counting conditions exhibit highly similar dynamics across all five PCs, suggesting shared underlying representational structures but differences in the precision of those representations. In contrast, the explicit counting condition reveals distinct spikes at multiples of five and decade numbers, indicating a more discretized and externally guided numerosity representation. Although the spontaneous counting condition is behaviorally similar to the mental and forbid conditions, its latent dynamics display a hybrid pattern. Specifically, for some target numerosities, spontaneous counting exhibits internal accumulator-like patterns similar to the mental and forbid conditions, while for others, it shows spike patterns aligned

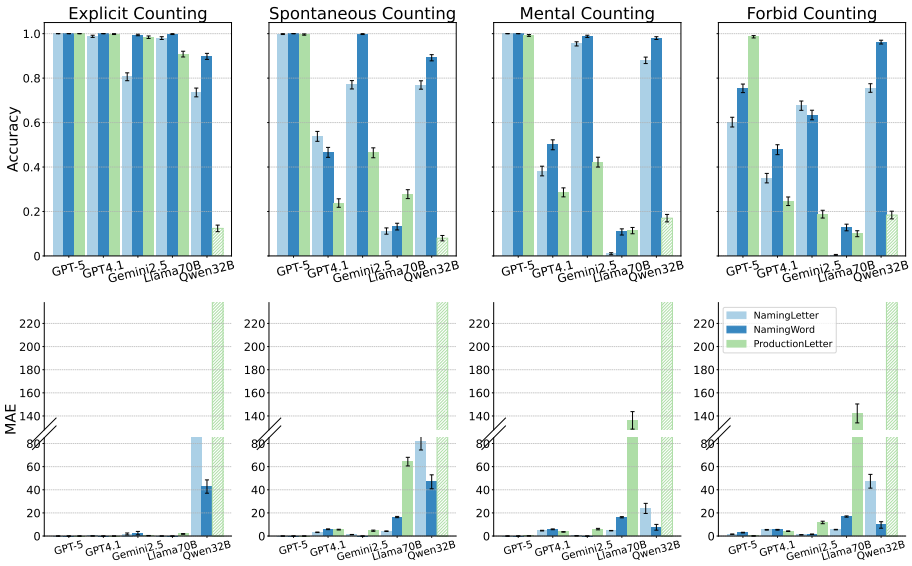

Fig. 8. Comparison of accuracy and mean absolute error (MAE) across different models and prompting conditions on heterogeneous stimuli. Bars represent accuracy or MAE for each model (x-axis) across the tasks. Error bars indicate the variability of the estimates: for accuracy, binomial standard errors are shown; for MAE, standard errors of the mean are shown. We flagged Qwen's results as it has a large number of invalid trials.

with those observed in the explicit condition. This suggests that spontaneous counting may engage a mixture of internal and externally anchored mechanisms in its representational trajectory.

Correlation Analysis of Population Encoding To quantify the relationship between individual neuron activations and sequence position, we computed Pearson correlation coefficients between each neuron's activation trajectory and the corresponding step indices across all trials in Section 4.3. Among the 8192 neurons, the top 500 with the strongest positive correlations exhibited a mean correlation coefficient of 0.523 ± 0.003 (SEM), while the top 500 with the strongest negative correlations had a mean of -0.527 ± 0.003 (SEM). In both cases, the average p-value across neurons was $< .001$, indicating a highly significant association between neural activity and sequential position.

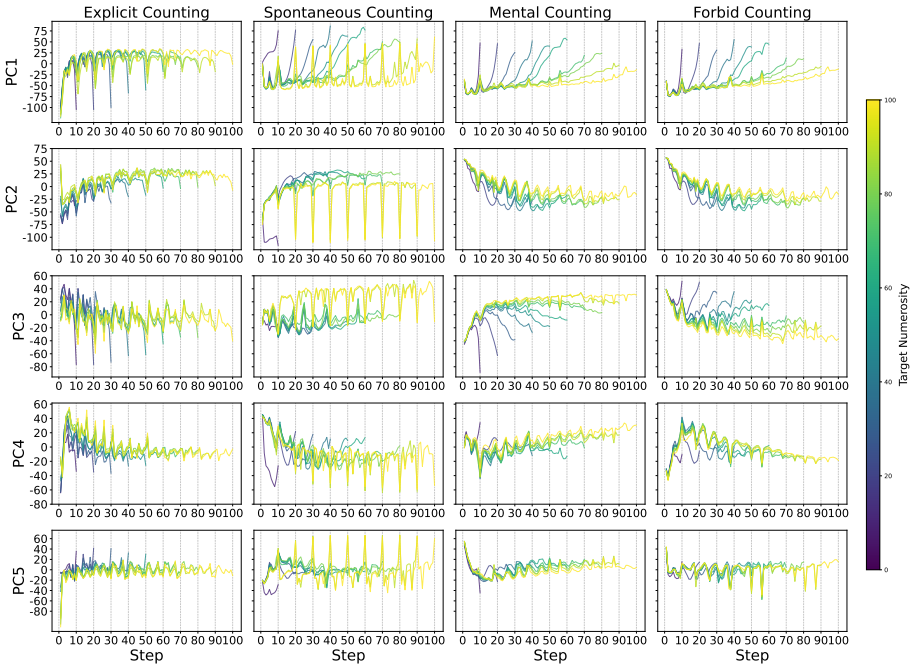

Fig. 9. Trajectories of the first five principal components computed over hidden states for all conditions. Each subplot shows the trajectories along the principal components as a function of generation step, with colors indicating the target numerosity.